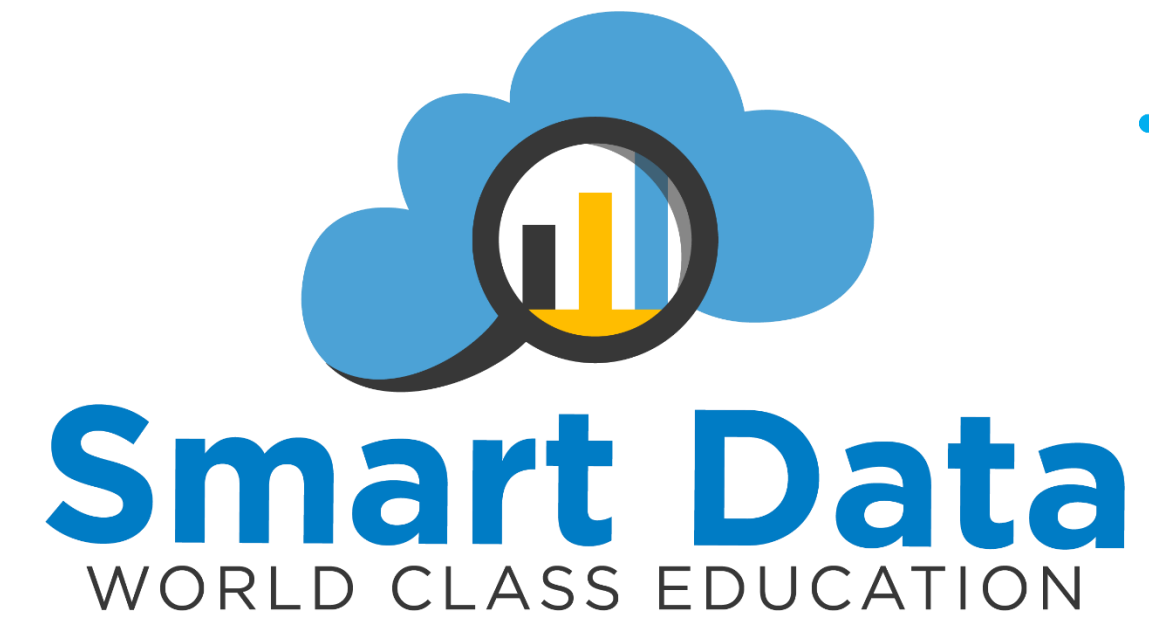
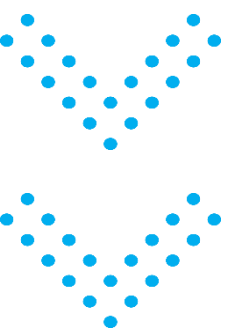
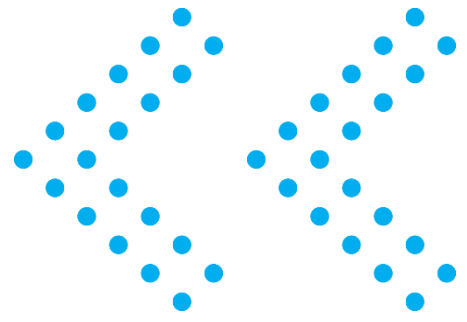
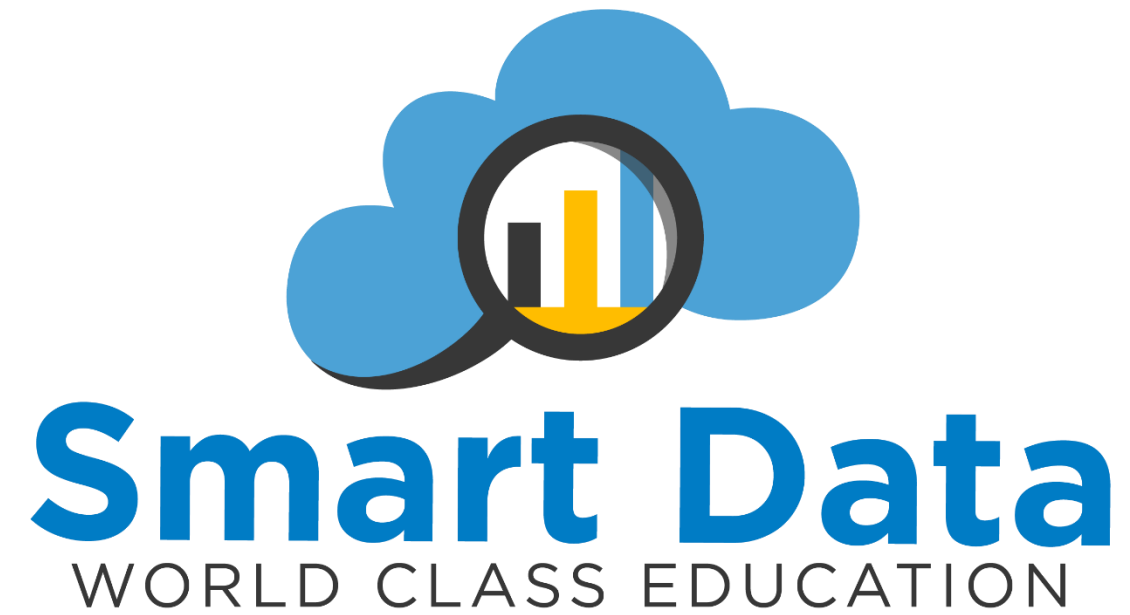




MLOps: Del Modelo al entorno Productivo



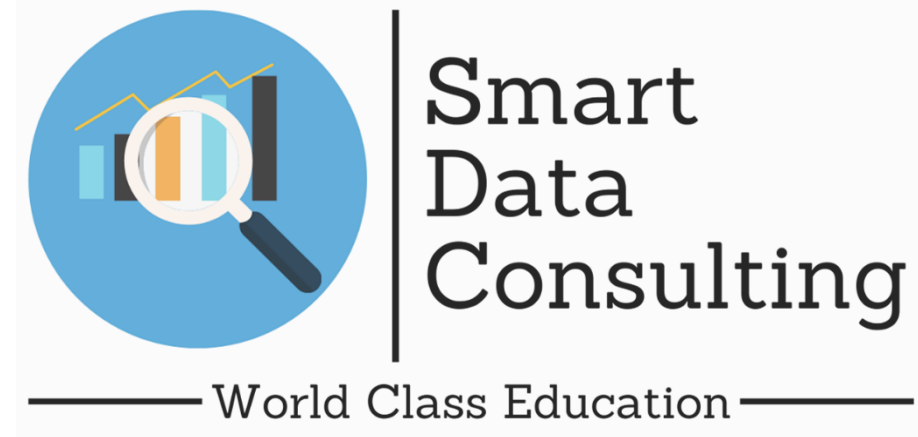


CI/CD básico aplicado a ML

Instructor:

Julio César Bernal Fernández

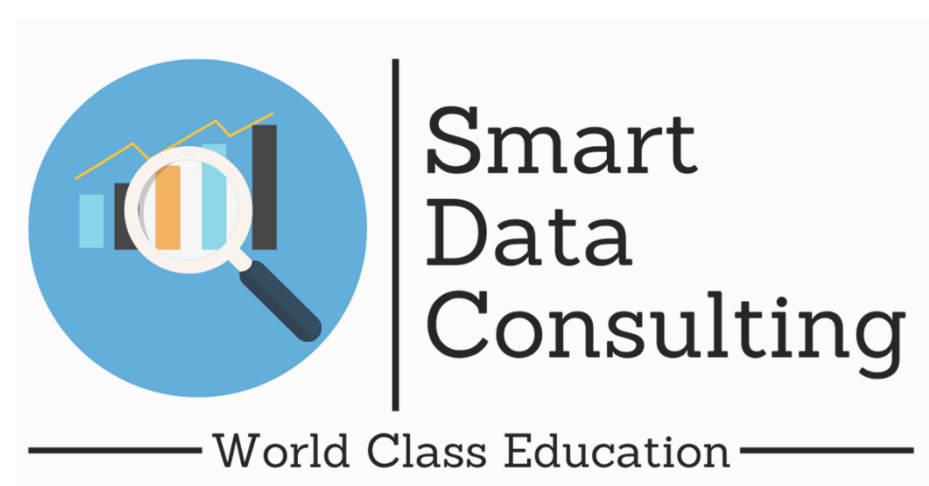
Integración Continua (CI)



La integración continua, o CI, como se conoce a menudo por su abreviatura en inglés, es la práctica de hacer que todos los que trabajan en el mismo proyecto de software compartan sus cambios en la base de código con regularidad, y luego comprobar que el código sigue funcionando como debería después de cada cambio. La integración continua constituye una parte fundamental del enfoque DevOps para la creación y lanzamiento de software, que promueve la colaboración, la automatización y los ciclos cortos de feedback.

La práctica de la integración continua comienza con la confirmación de los cambios en un sistema de control de versiones/fuentes con regularidad, para que todos construyan sobre la misma base. Cada confirmación desencadena una compilación y una serie de pruebas automatizadas para verificar el comportamiento y asegurar que el cambio no ha estropeado nada. Aunque la integración continua es beneficiosa por sí misma, también es el primer paso hacia la implementación de un proceso de CI/CD.

Entrega Continua (CD)



La entrega continua comienza donde acabó la integración continua. En la etapa que sigue a la creación de un proceso de CI/CD, la entrega continua automatiza los pasos necesarios para entregar su software a la producción, pero no llega a ser un proceso de lanzamiento totalmente automatizado.

¿Qué es la entrega continua?

El objetivo de la entrega continua es hacer que el proceso de lanzamiento de un software sea más rápido y más fiable, abreviar el tiempo que se tarda en obtener feedback y aportar valor a los usuarios de un modo más rápido que con un proceso manual.

Una vez que los lanzamientos son estables y reproducibles, resulta sencillo lanzar más a menudo, y puede comenzar a incorporar pequeñas mejoras cada semana, cada día o incluso cada hora. Al igual que la integración continua, la entrega continua requiere de la trinidad de DevOps (Desarrollo y operaciones) para su puesta en práctica: herramientas, proceso y cultura.

Implementación Continua (CD)

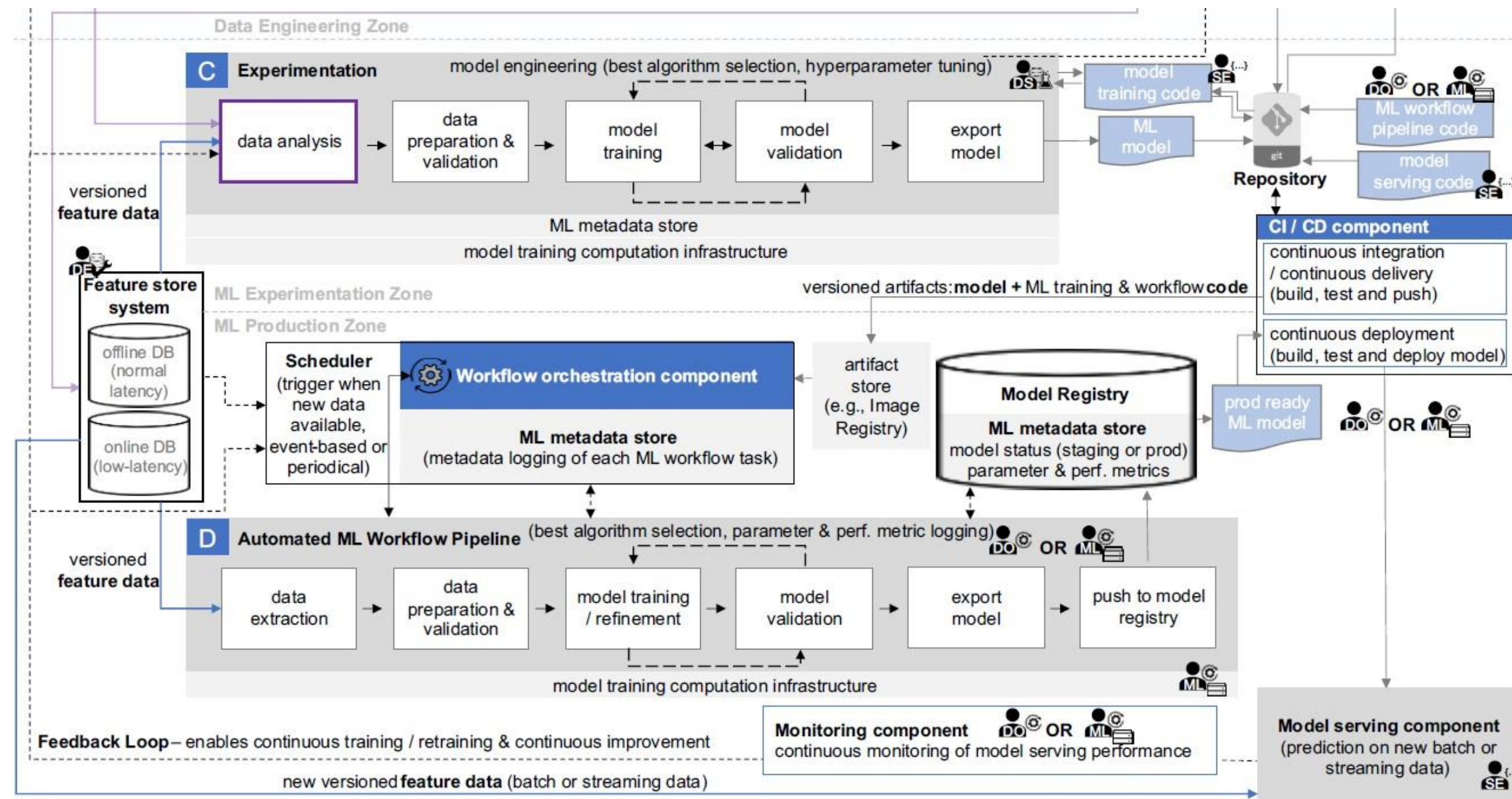


La implementación continua lleva a su extremo lógico la práctica de DevOps de automatizar los pasos de compilación, pruebas e implementación. Si un cambio en el código pasa correctamente por todas las etapas previas del proceso, ese cambio se implementa automáticamente en la producción sin intervención manual alguna. Adoptar la implementación continua significa que puede hacer llegar las funcionalidades a los usuarios del modo más rápido posible sin poner en entredicho la calidad.

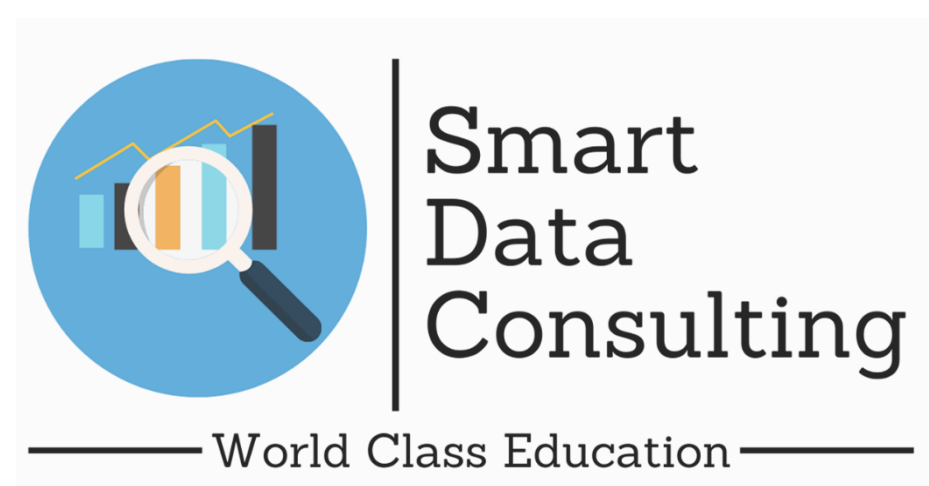
La implementación continua se apoya en una integración continua demostrada y madura, así como en las etapas de entrega continua. Los pequeños cambios en el código se confirman con regularidad para pasar a master, pasan por un proceso de compilación y prueba automatizado con avance a través de varios entornos de preproducción y, si no se detecta ninguna incidencia, por último se lanzan al usuario. Crear un proceso de implementación automatizado que sea estable y fiable significa que un lanzamiento pueden convertirse en algo rutinario que sucede varias veces al día.

CI/CD para Data Science

- En Ciencia de Datos, los conceptos de Integración y Entrega Continua se relacionan con la posibilidad de versionar los códigos y automatizar los tests. En cuando a Despliegue Continuo, se refiere a simplificar las implementaciones en la infraestructura productiva.



Soluciones CI/CD



- [Jenkins](#)

Más genérico y orientado al despliegue continuo, sin embargo también puede servir para construir un [ecosistema de implementación](https://docs.microsoft.com/es-es/azure/developer/jenkins/pipeline-with-github-and-docker). Por ejemplo: <https://docs.microsoft.com/es-es/azure/developer/jenkins/pipeline-with-github-and-docker>

- [Github Actions](#)

Amplia las capacidades de Github para crear ambientes virtuales en los que podemos ejecutar las aplicaciones o desplegar contenedores

- [SVC](#)

Permite versionar Datos y Código, ampliando las capacidades de Git

- [CML](#)

Permite desplegar flujos de trabajo en contenedores para

Registro de Modelos y Gestión de Metadatos

Registro de Modelos y Gestión de Metadatos



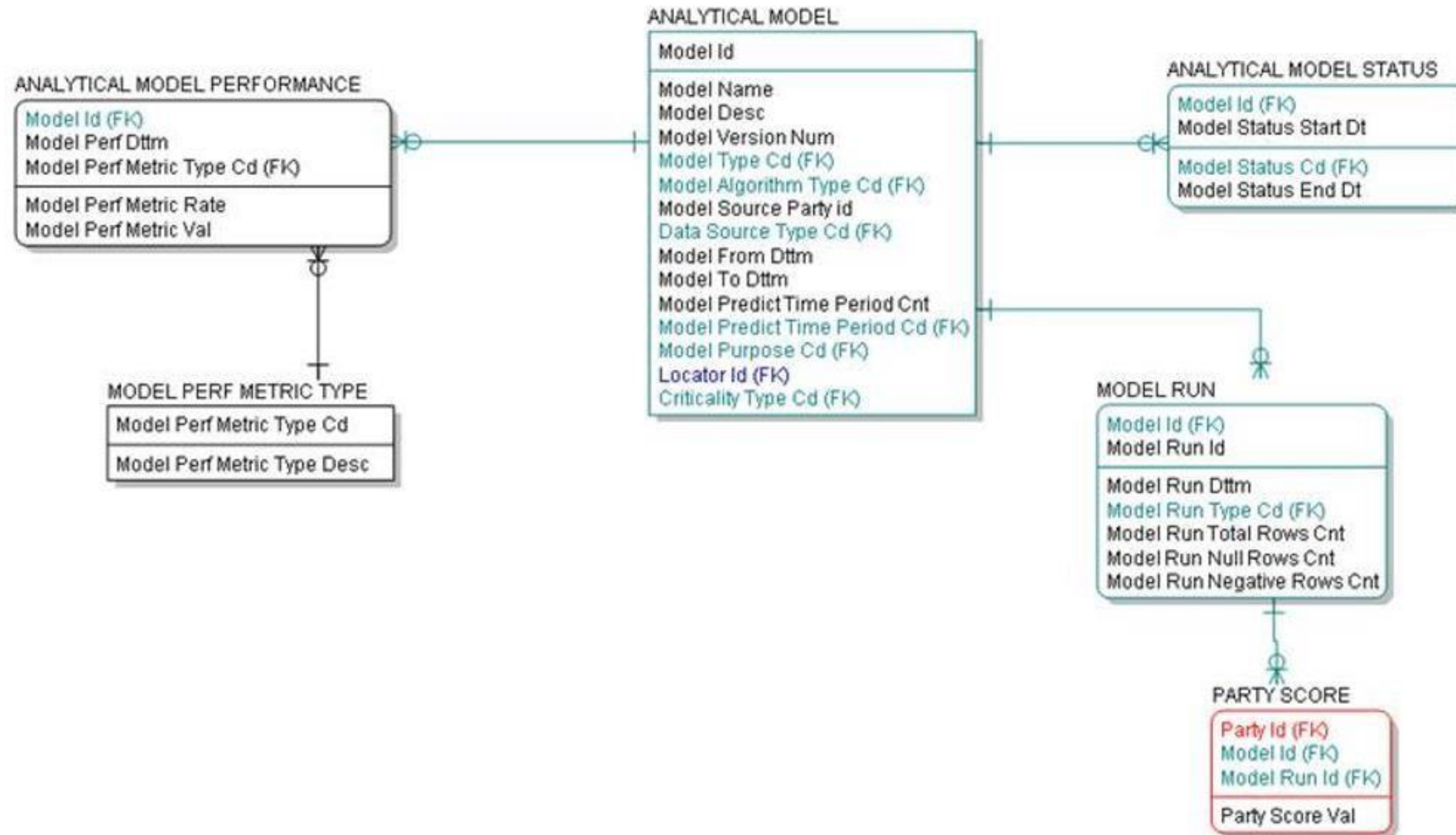
Estas soluciones permiten completar la parte administrativa de la gestión de modelos.

Registro de Modelos: Mecanismo que nos permite identificar los modelos que existen en el entorno productivo, el estado de cada uno y particularmente, el proceso de alta, aprobación, actualización, retiro, vigencia, etc.

Gestión de Metadatos: Permite almacenar los resultados de las ejecuciones de los modelos, tanto en la fase de evaluación y scoring, como en la fase posterior, cuando se necesita comparar los datos de las predicciones con los valores reales.

Modelo de Datos y Metadatos

Ejemplo de un modelo de Datos y Metadatos



Model Development Lifecycle

Verta.ai

comet

SNARK

Polyaxon

mlflow

DVC

sacred

Pachyderm

datmo

Deep Learning



Determined AI



Weights & Biases



missinglink.ai

Model Governance

mlflow



datatron

ALGORITHMIA



open data



John Snow LABS

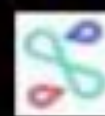
ParallelM

Model Operations, Testing, and Monitoring

ParallelM



John Snow LABS



SELDON



datatron

Deep Learning

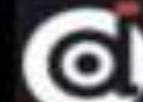


Determined AI

PipelineAI



hydrosphere.io



open data

ALGORITHMIA

Verta.ai



SNARK



Ray Serve



Kubeflow

TFX

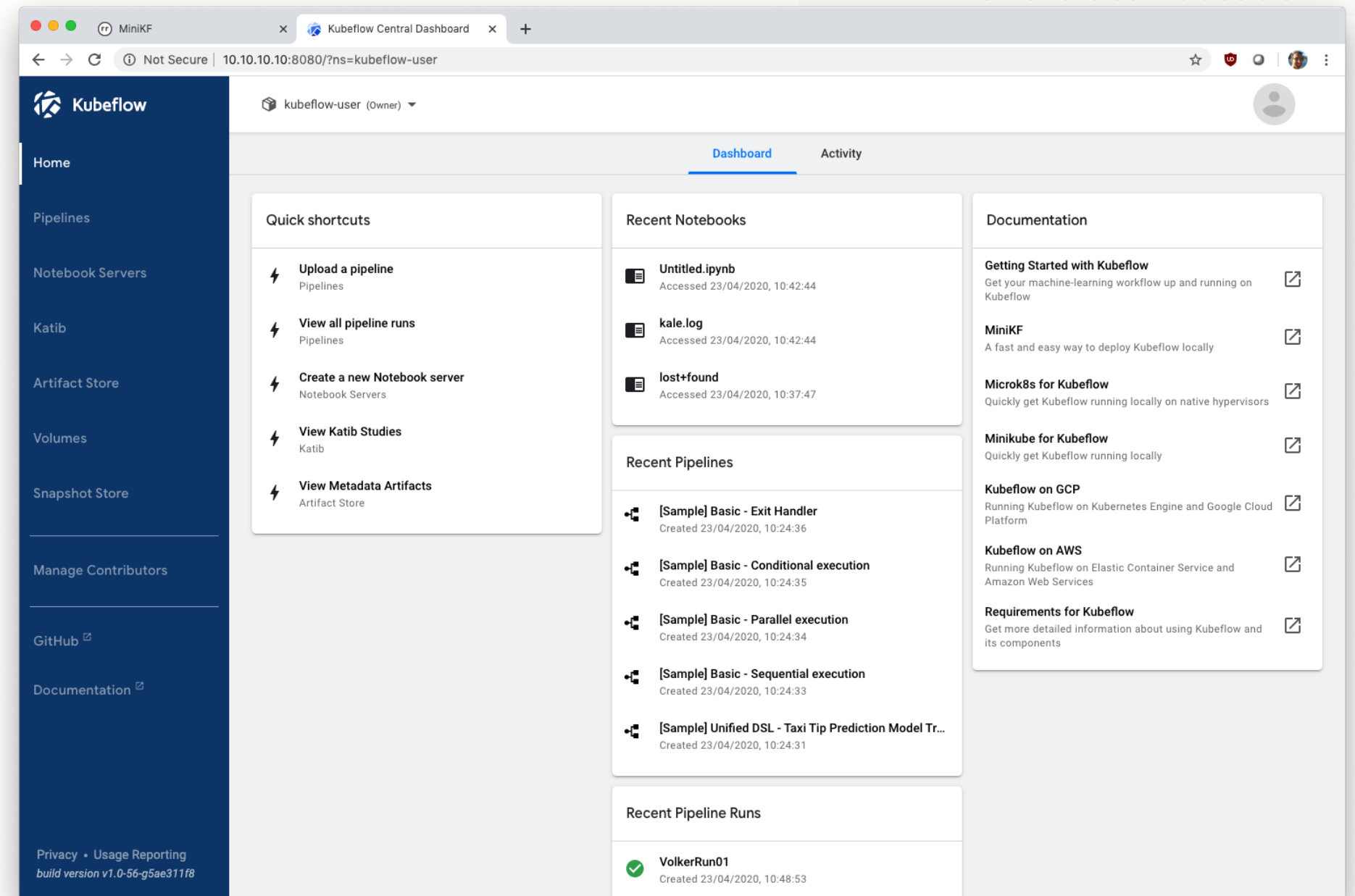
KubeFlow

Kubeflow is essentially a self-hosted version of the Google AI platform. It uses Kubernetes to abstract away infrastructure.

Kubeflow can deploy Jupyter notebooks, run pipelines for data processing and model training (scheduled, on-demand), organize runs, archive models and other artifacts, and expose models through endpoints. Pipelines are compute graphs and are described in Python with a DSL. Their components are wrapped as Docker images.

It integrates with GCP so it can elastically scale to the cloud compute and storage (e.g., distributed model training). It also integrates with offerings such as BigQuery or Dataproc.

The solution is heavy and complex but enables rapid scale-out. It is especially applicable if infrastructure is already managed through Kubernetes.



<https://www.kubeflow.org/docs/started/workstation/getting-started-minikf/>

```
$ cd /somewhere
$ vagrant init arrikto/minikf
$ vagrant up
---> http://10.10.10.10
```

Neptune

Neptune is a tracker for experiments with focus on Python. It is a hosted service.

Experiments are tracked by using library hooks to register (model) parameters, evaluation results, and upload artifacts (such as models, hashes of training data, or even code). The library can track hardware usage and experiment progress.

The results can be analyzed and compared on a website. There are also collaborative options. Neptune has integrations with Jupyter notebooks, various ML libraries, visualizers (HiFlow, TensorBoard), other trackers (MLFlow), and external offerings (Amazon Sagemaker).

They provide an API to query results from experiment. This can be used to feed CI/CD pipelines for model deployment.

A screenshot of the Neptune web interface. The browser address bar shows 'ui.neptune.ai/vhffm/sandbox/experiments?viewId=standard-view'. The interface has a dark header with 'Projects' and 'Explore' tabs. Below the header, there's a navigation bar with 'Experiments' (9), 'Notebooks' (0), 'Wiki', and 'Settings'. A search bar and several filters (Time, Tags, Experiment Status, Owned By) are present. A table lists experiments with columns for ID, State, Name, Created, Finished, Running, and various metrics like decay_factor, n_iterations, AUC, and loss. The experiments are sorted by ID, showing SAN-9 down to SAN-1. Most are 'SUCCEEDED', but SAN-6 and SAN-5 are 'FAILED'.

ID	State	Name	Created	Finished	Running	decay_fac...	n_iterations	AUC	loss	Tags
SAN-9	SUCCEEDED	ex6	17 APR 10:...	17 APR 10:...	< 1 min	0.5	117	0.96	0.00623214	tag1 tag2
SAN-8	SUCCEEDED	ex5	17 APR 10:...	17 APR 10:...	< 1 min	0.5	117	0.96	0.00623214	
SAN-7	SUCCEEDED	ex4	17 APR 10:...	17 APR 10:...	< 1 min	0.5	117	0.96	0.00623214	
SAN-6	FAILED	ex4	17 APR 10:...	17 APR 10:...	< 1 min	0.5	117	0.96	0.00623214	
SAN-5	FAILED	ex4	17 APR 10:...	17 APR 10:...	< 1 min	0.5	117	0.96	0.00623214	
SAN-4	SUCCEEDED	ex3	17 APR 10:...	17 APR 10:...	< 1 min	0.5	117	0.96	0.00623214	
SAN-3	SUCCEEDED	ex2	17 APR 10:...	17 APR 10:...	< 1 min	0.5	117	0.96	0.00623214	
SAN-2	SUCCEEDED	minimal_example	17 APR 10:...	17 APR 10:...	< 1 min	-	-	0.96	0.00623214	
SAN-1	SUCCEEDED	minimal_example	17 APR 10:...	17 APR 10:...	< 1 min	-	-	0.96	0.00623214	

```
$ conda create --name neptune python=3.6
$ conda activate neptune
$ conda install -c conda-forge neptune-client
$ cd /somewhere ---> https://ui.neptune.ai/
---> Create Account, Log In, "Getting Started"
```

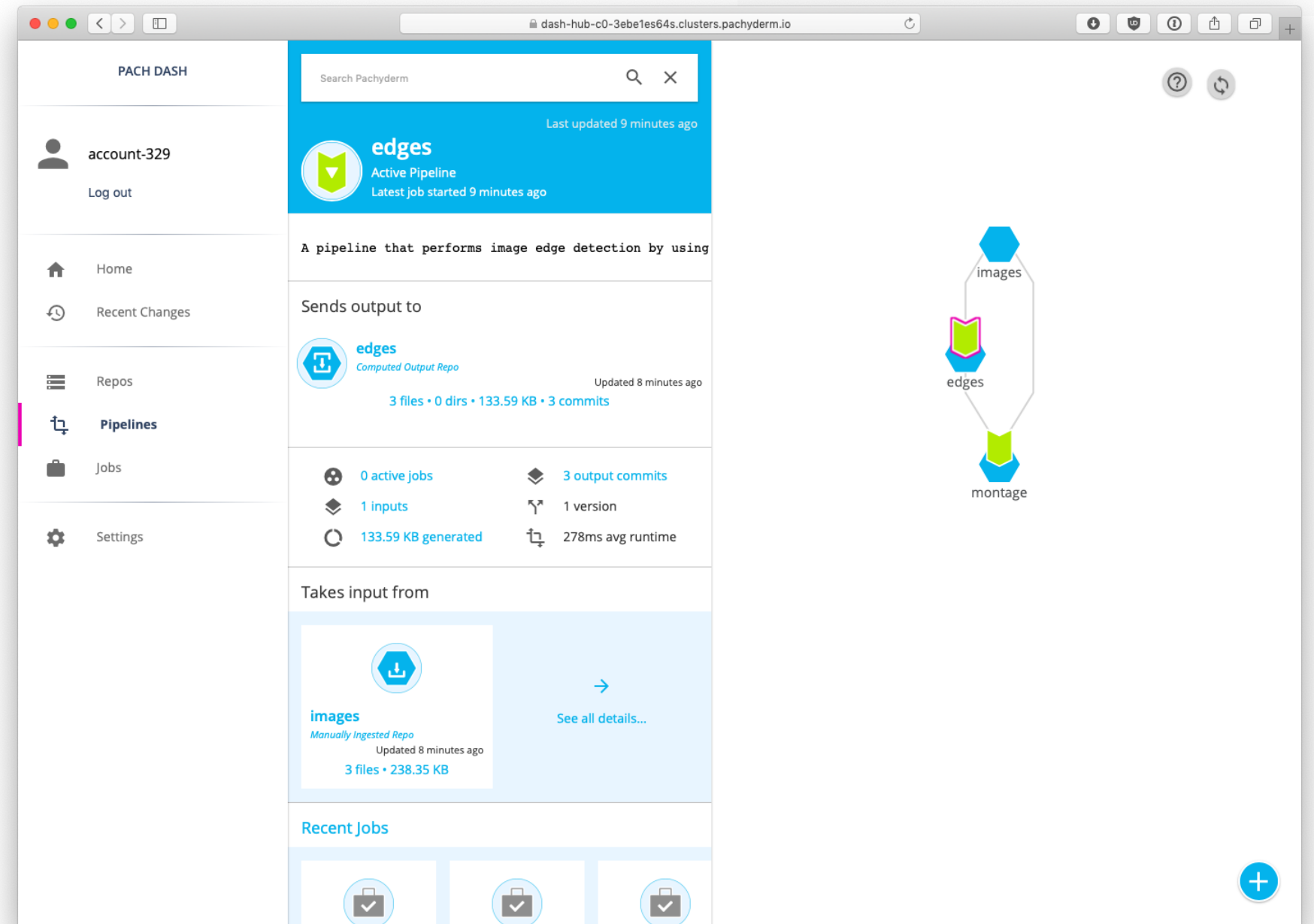
Pachyderm

Pachyderm is a versioning system and execution environment for data and processing pipelines. Hosted and self-hosted options exist.

At its core is data provenance. Data is committed to a repository and acted upon by processors in pipelines. The results (data and other artefacts like models) are committed back into a repository.

By design, all use of data is traceable through pipelines. Pipelines are described in JSON and processors are packaged as Docker images. Pachyderm can integrate (but not deploy) Jupyter and can push/pull data from cloud stores (S3, etc).

Pachyderm is built on top of Kubernetes, so can easily scale-out and run in various clouds. Self-hosting comes with the usual Kubernetes complexity.



```
$ cd /somewhere
$ download from (https://github.com/pachyderm/pachyderm/releases)
$ tar xfvz release_filename_linux_amd64.tar.gz
$ pachctl version --client-only
--> https://docs.pachyderm.com/latest/pachub/pachub\_getting\_started/
--> https://docs.pachyderm.com/latest/getting\_started/beginner\_tutorial/
```

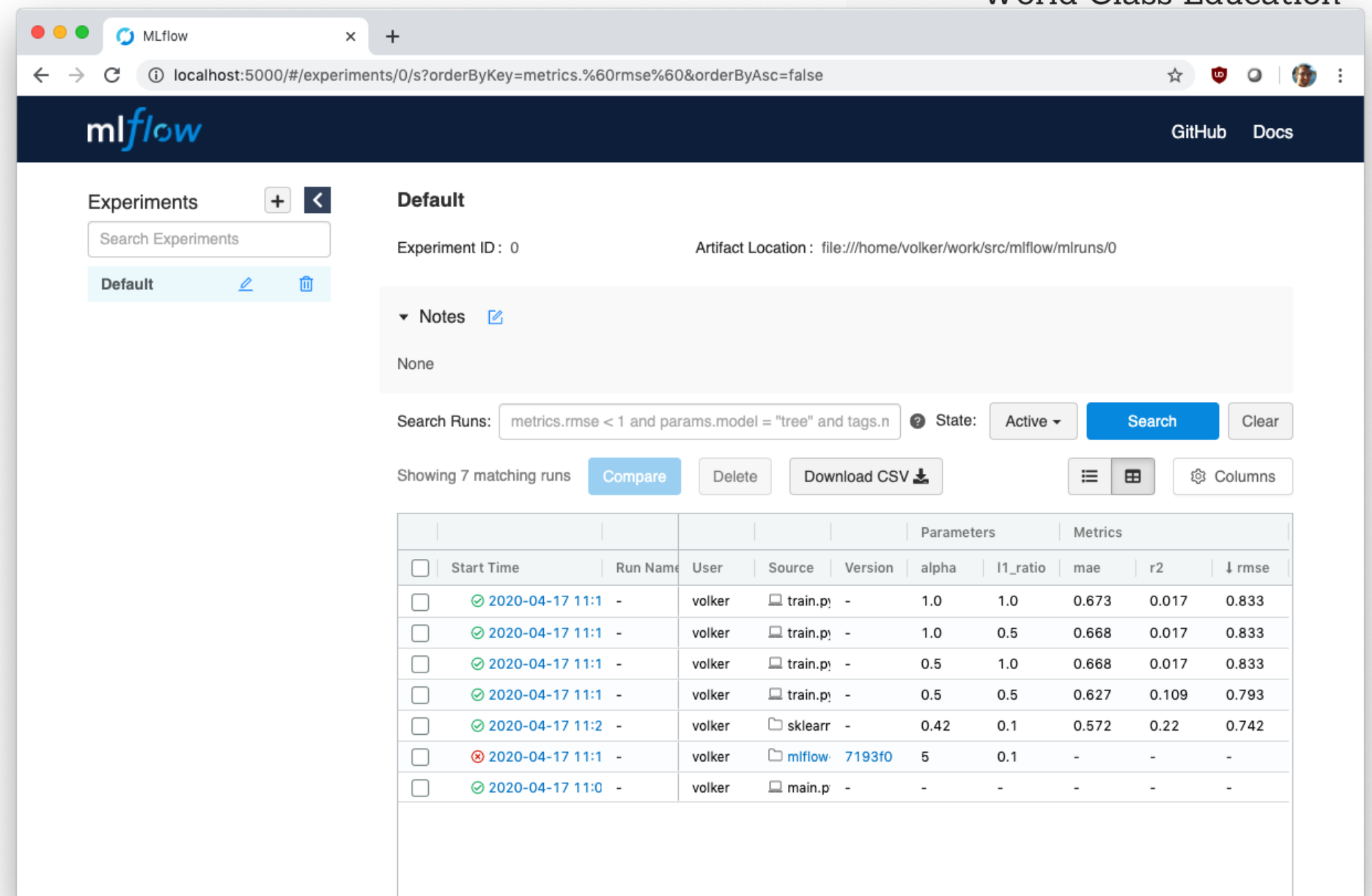

MLflow

MLflow is an experiment tracker and generic model server. It can be self-hosted.

Experiments are tracked by using library hooks to register (model) parameters, evaluation results, and upload artifacts (such as models, hashes of training data, or even code). Artifacts can be logged to local, remote, or cloud storage (S3, GFS, etc).

Results can be analyzed through a web UI and CSV export is available. Models are packaged as a wrapper around the underlying format (Sklearn, XGBoost, Torch, etc). They can be pushed to Spark for batch inference or served through REST.

There are CLI, Python, R, Java, and REST APIs for further integration with CI/CD pipelines. Models can be pushed to cloud services (SageMaker, AzureML, ...).



```
$ conda create --name mlflow python=3.6
$ conda activate mlflow
$ conda install -c conda-forge mlflow
$ cd /somewhere
$ mlflow ui ---> http://localhost:5000
https://mlflow.org/docs/latest/quickstart.html
```

Características de MLflow

- Interfaz abierta y código abierto: MLflow está diseñado para funcionar con cualquier biblioteca, algoritmo, herramienta de implementación o lenguaje de ML. También permite compartir pasos y modelos del flujo de trabajo entre distintos equipos.
- Se basa en API REST y formatos de datos simples (p. ej., un modelo se puede ver como una función lambda) que se puede usar desde una variedad de herramientas, en lugar de solo proporcionar un pequeño conjunto de funciones integradas.

mlflow

Tracking

Record and query experiments: code, data, config, results

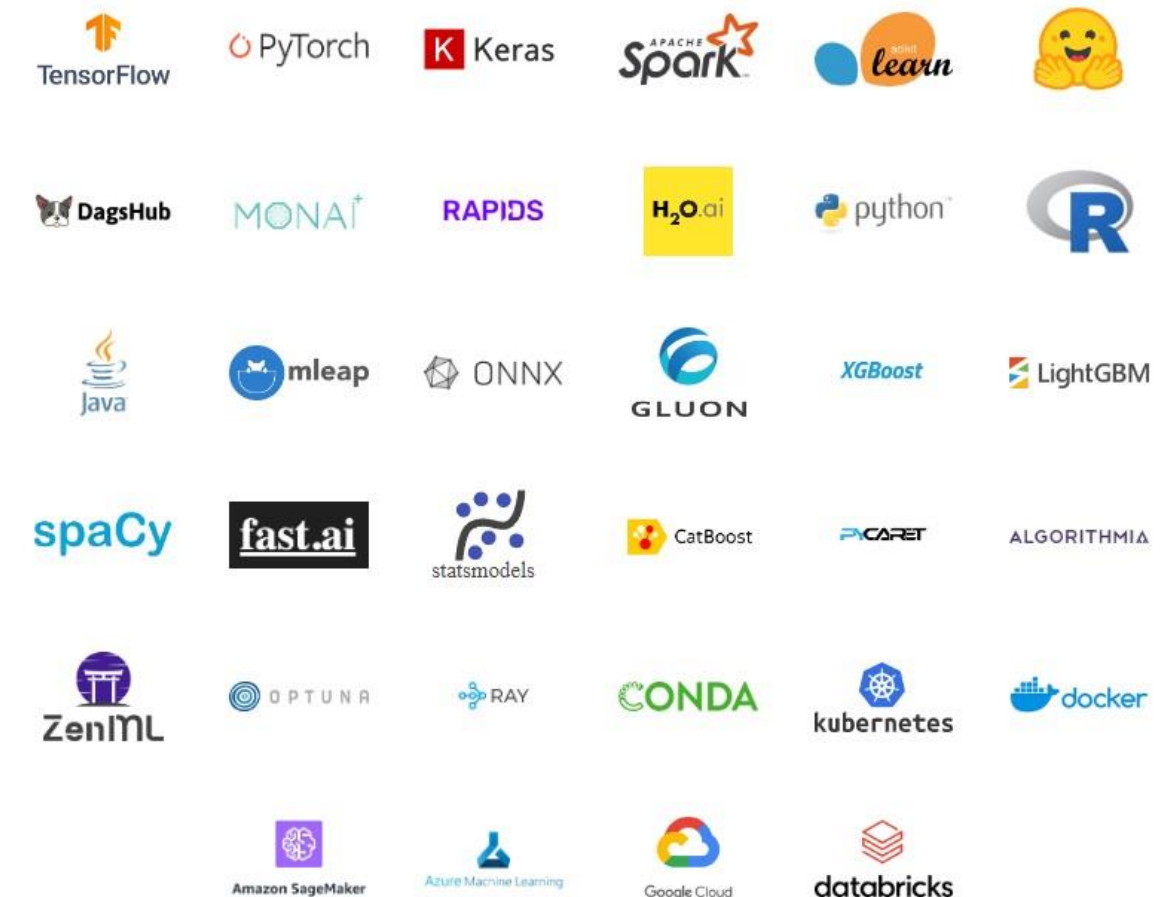
Projects

Packaging format for reproducible runs on any platform

Models

General format for sending models to diverse deploy tools

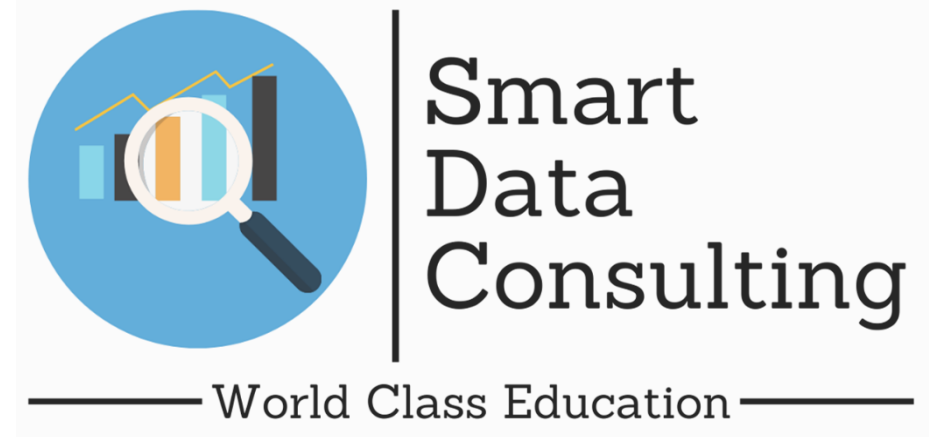
Integraciones con:



Seguimiento de Modelos en MLflow

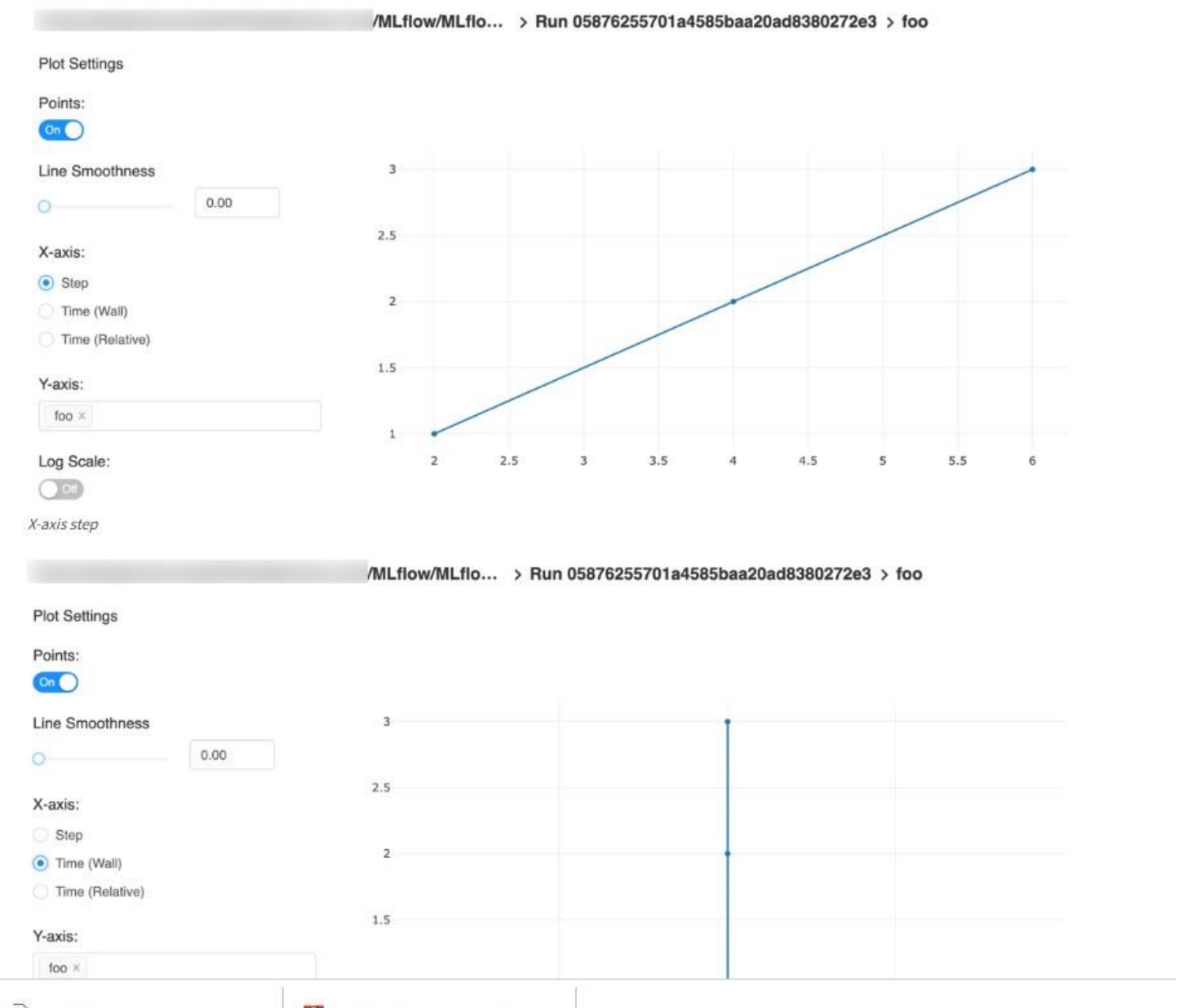
Seguimiento de flujo de aprendizaje automático

MLflow Tracking es una API y una interfaz de usuario para registrar parámetros, versiones de código, métricas y archivos de salida al ejecutar su código de aprendizaje automático para visualizarlos más tarde. Con unas pocas líneas de código simples, puede rastrear parámetros, métricas y artefactos:



Visualizing Metrics

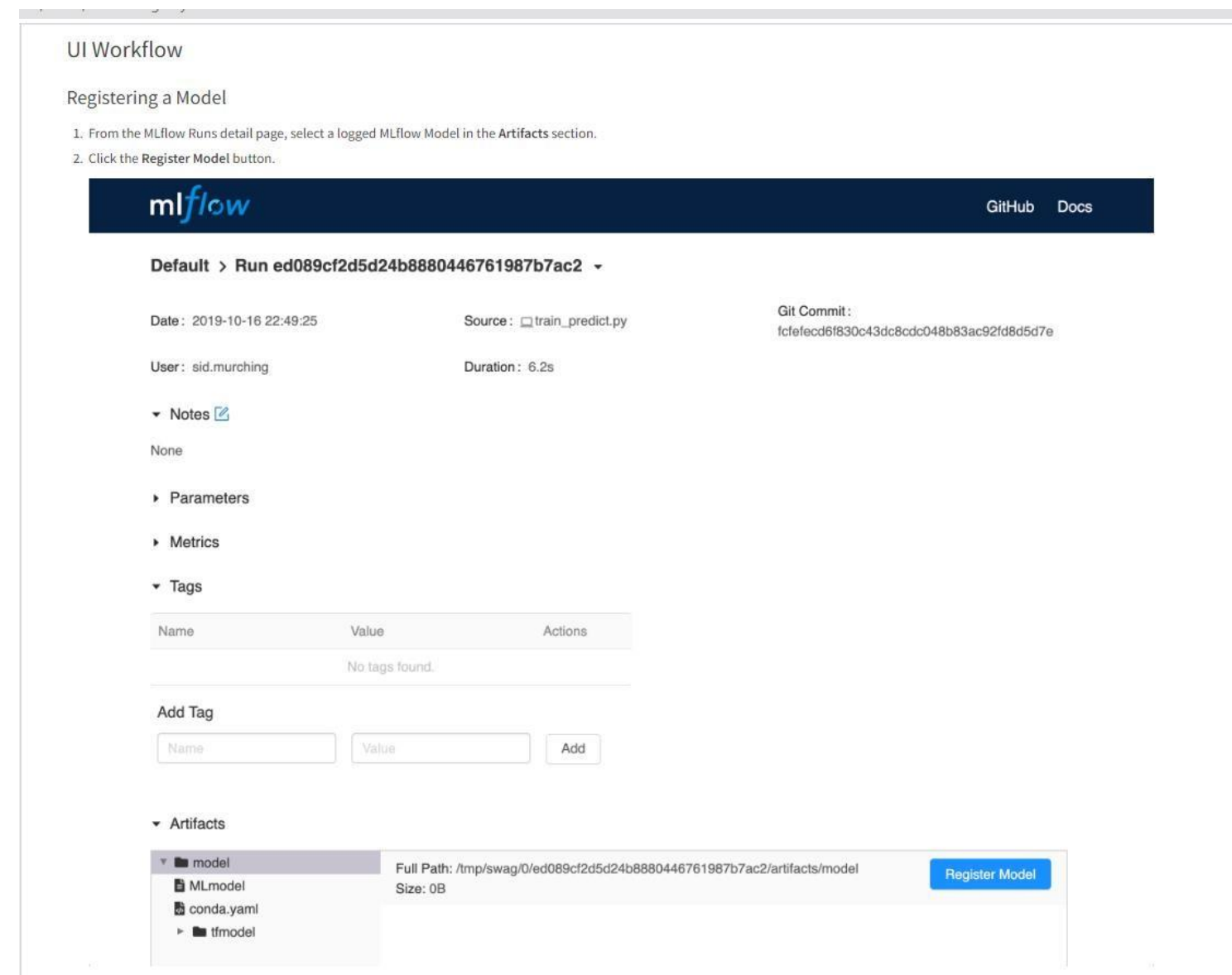
Here is an example plot of the quick start tutorial with the step x-axis and two timestamp axes:



Registro de Modelos en MLflow

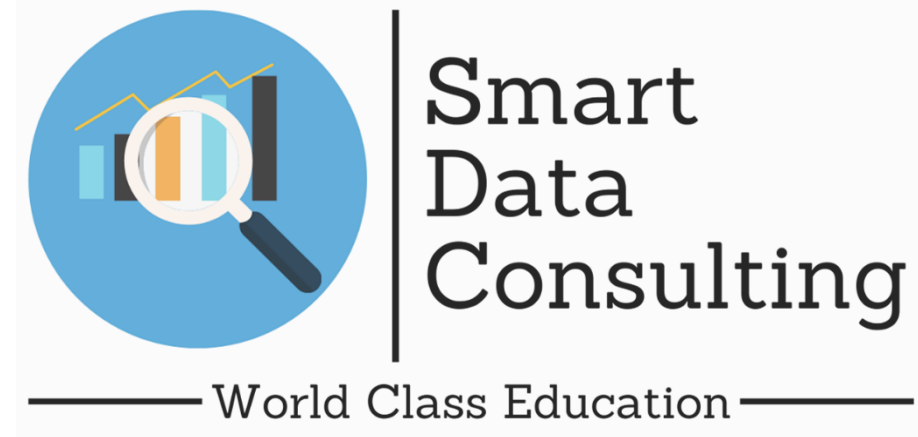


- Almacene, anote, descubra y administre modelos en un repositorio central.
- Cada modelo de MLflow se guarda como un directorio que contiene archivos arbitrarios y un MLmodelarchivo descriptor que enumera los sabores en los que se puede usar.



```
time_created: 2018-02-21T13:21:34.12
flavors:
  sklearn:
    sklearn_version: 0.19.1
    pickled_model: model.pkl
python_function:
  loader_module: mlflow.sklearn
  pickled_model: model.pkl
```

Proyectos de MLflow



Los proyectos de MLflow proporcionan un formato estándar para empaquetar código de ciencia de datos reutilizable. Cada proyecto es simplemente un directorio con código o un repositorio Git y utiliza un archivo descriptor para especificar sus dependencias y cómo ejecutar el código. Un proyecto MLflow se define mediante un archivo YAML simple llamado MLproject.

name: My Project

```
name: My Project
conda_env: conda.yaml
entry_points:
  main:
    parameters:
      data_file: path
      regularization: {type: float, default: 0.1}
    command: "python train.py -r {regularization} {data_file}"
  validate:
    parameters:
      data_file: path
    command: "python validate.py {data_file}"
```

Proyectos de MLflow

Proyectos de MLflow

Empaquete el código de ciencia de datos en un formato para reproducir ejecuciones en cualquier plataforma

MLproject File

You can get more control over an MLflow Project by adding an MLproject file, which is a text file in YAML syntax, to the project's root directory. The following is an example of an MLproject file:

```
name: My Project

python_env: python_env.yaml
# or
# conda_env: my_env.yaml
# or
# docker_env:
#   image: mlflow-docker-example

entry_points:
  main:
    parameters:
      data_file: path
      regularization: {type: float, default: 0.1}
    command: "python train.py -r {regularization} {data_file}"
  validate:
    parameters:
      data_file: path
    command: "python validate.py {data_file}"
```

```
import mlflow
import numpy as np
from sklearn.linear_model import LogisticRegression

with mlflow.start_run():
    X = np.array([-2, -1, 0, 1, 2, 1]).reshape(-1, 1)
    y = np.array([0, 0, 1, 1, 1, 0])
    lr = LogisticRegression()
    lr.fit(X, y)

    model_info = mlflow.sklearn.log_model(sk_model=lr, artifact_path="model")

sklearn_pyfunc = mlflow.pyfunc.load_model(model_uri=model_info.model_uri)

data = np.array([-4, 1, 0, 10, -2, 1]).reshape(-1, 1)

predictions = sklearn_pyfunc.predict(data)
```